Capitolo 4

Week 17-23 November

4.1 Channel models (frequency selective, FIR, flat channel); channel normalization and average symbol energy; vector representation of a SISO FIR channel

An important thing is the normalization of the channel; people normally make lots of mistakes about this.

We have the model defined in (2.19). We can model such a channel as

$$c_p(\tau) = \sum_{q=0}^{Q-1} A_q \delta(\tau - \tau_q) \ . \tag{2.19}$$

If we don't pay attention, this channel could be amplified. We need to pay attention because the energy of the channel is

$$\sum |A_q|^2 \ ,$$

which might be greater than 1.

> What's important is the ratio between the noise and the amplification of the channel (which comes from the antenna itself, its directionality etc etc): the gain is in respect to an isotropic antenna.

In short, the problem is that we want to separate the gain from the channel response, which is why we write:

$$c(\tau) = \sqrt{G} h(\tau) \tag{2.64}$$

$$c_b(\tau) = \sqrt{G} h_b(\tau) \tag{2.65}$$

$$c[l] = \sqrt{G} h[l] \tag{2.66}$$

we'll also impose that:

$$\sum_l \mathbb{E}|h[l]|^2 = 1 \tag{4.1}$$

the channel gain should always be one.

Rem: This makes perfect sense. *We know that a generic request on the channel is that*

$$\sum_l \mathbb{E}\|H[l]\|_F^2 = N_t N_r$$

here we're in SISO, thus we have only one receiver and one transmitter; it makes perfect sense to request that the sum of the expectations is 1.

Def 4.1.1: Frequency selective model. *The **frequency selective** is a model in which we have $\sqrt{G}$ then a causal and FIR (Finite Impulsive Response) channel and then noise which is IID and $v[n] \sim \mathcal{N}_{\mathbb{C}}(0, N_0)$.*

The output of a generic channel, assuming the normalization we mentioned holds, would be:

$$y[n] = \sqrt{G}\sum_l h[l]x[n-l] + v[n] \tag{2.69}$$

If the condition of FIR and of causality hold, though, the output can be simplified as:

$$y[n] = \sqrt{G}\sum_{l=0}^{L} h[l]x[n-l] + v[n] \tag{2.70}$$

If we then have $L = 0$, the channel is multiplicative and will be called **flat channel**, giving an output of:

$$y[n] = \sqrt{G}h[0]x[n] + v[n] \tag{2.71}$$

all the frequencies are treated the same; we can have this model represent a time variant setup if $h[0]$ actually depends on n.

If $L > 0$ the channel exhibits ISI (Inter-Symbol Interference) and since its response will not be flat in frequency, we say that it is **frequency-selective**.

Symbols are normalized to have an **average symbol energy**:

$$E_s = \frac{1}{N}\sum_{k=0}^{N-1} \mathbb{E}|x[k]|^2 \tag{4.2}$$

We'll see that this specific definition is just the per-block power constraint. We can also have per-symbol or per-antenna power constraints, from which will follow other constraints on the precoding matrix (see signal power constraints).

This will allow us to write the SNR as:

$$\text{SNR} = \frac{P_r}{N_0/T} = \frac{GE_s}{N_0}, \tag{4.3}$$

where:

- P_r is the power of the received signal, which in turn is equal to $P_r = GP_x = GE_s/T$ (P_x is the power of the transmitted signal);
- N_0 is the covariance of the AWGN.

This last relation will turn out to be useful later on (it is literally (4.2), (4.3) and it will be of use to write (4.6), (4.7)...).

If we have a frequency-selective channel, we can give a vector representation of (2.70). For the channel we'll use a Toeplitz matrix, which is a $N \times (N+L)$ convolution matrix defined as:

$$H_{N,N+L} = \begin{bmatrix} h[L] & ... & h[0] & 0 & ... & 0 \\ 0 & h[L] & ... & h[0] & 0 & \vdots \\ \vdots & & \ddots & & \ddots & 0 \\ 0 & ... & 0 & h[L] & ... & h[0] \end{bmatrix} \tag{2.72}$$

the input signal is consequently defined as:

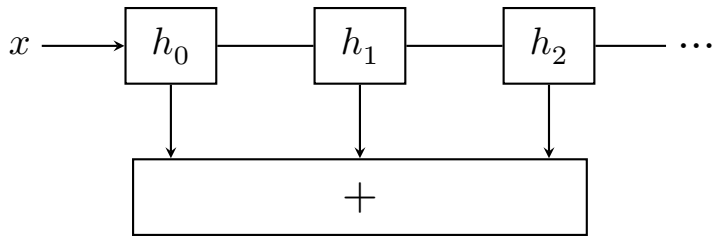
$$x_{N+L} = \begin{bmatrix} x[-L] \\ \vdots \\ x[-1] \\ x[0] \\ x[1] \\ \vdots \\ x[N-1] \end{bmatrix} \tag{2.73}$$

giving that if the channel noise is v_N, we'll have:

$$y_N = [y[0] \, ... \, y[N-1]]^T = \sqrt{G} H_{N,N+L} x_{N+L} + v_N \tag{2.74}$$

We can show that whichever n-th row in (2.74) is equivalent to (2.70).

This is like if we had a MIMO system: "Vectorized result makes an equivalence between frequency-selective SISO channels and frequency-flat MIMO channel, and may be useful for deriving estimators or equalizers".



Block diagram of the FIR/Channel model (reconstruction)

This is the same of having a MIMO channel with different gains. We can ask: if we want to estimate a symbol, how can we only get the symbol $x[n]$ we're interested in from $y[n]$?

We'll want to massage the MIMO system to make it similar to our model; we can put together all the matrices and get one only matrix: this is the stacked vector representation.

4.2 Organization detour (+mentions of SVD, Precoding)

Organizational issue: he set up a lecture for next wednesday; this is because on the 1st of November we'll have no lecture. Professor tried to keep the target so that we could finish before christmas. We'll also have some other lectures to look over.

One of the next topics is OFDM; he suggests to read the part of the book dealing with it before going through it with lectures; if we read it beforehand, we'll be in a better position.

Lore of the course moment About MMSE: when we transmit, this is another way to estimate the symbol transmitted.

Blabbering ab MIMO: if we have N tx and M rx, we have NM paths.

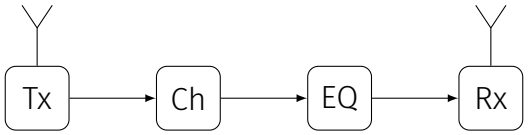
We'll see filters whose coefficients are matrices.

Singular value decomposition: we'll have a specific slide on this.

Precoding: we should try, if possible, to figure what a certain equation means in practice; the antenna may have a main lobe and we can change the main lobe by moving the antenna mechanically. Precoding is changing the type of transmission without moving the antenna mechanically; we do this using a certain matrix which we call precoding. With precoding we can steer where the antennas are transmitting.

We'll move to OFDM (used for 4G, 5G and probably will also be used for 6G); we need to see why we use OFDM! That's the wrong approach, since there are other types of modulation (single carrier modulation, just as an example). When we're transmitting, we have two antennas and a single channel.

We model the channel via spans:



If we have reflection, the previous signal might superpose to the previous one. The channel has a non-linear response: we can equalize it! We have many ways to equalize the channel!

The EQ in MIMO would increase the noise! We thus look at two **different types of equalizations**; we can have a linear or a non-linear equalization, but the latter is more resource-heavy.

We'll then look at the basics of radio propagation and such; there will be many concepts which are really important and will be pointed out. We'll see the Friis equation.

◇ Exam

Example of exam question: we can transmit at 900MHz or at 24GHz; we have a fixed power P and we can decide which frequency.
Thanks to the Friis equation, we know that choosing the lower frequency is better.

The state is getting more money from choosing the higher frequency.

4.3 Problems in discretization: multiple paths, multiple taps (+synchronization, reflection and noise whitening)

We've seen that long time ago people were manipulating analog signals, aka continuous time. Microphones are still in frequency modulation, as an example. Long time ago, we were able to communicate and we supposedly went to the moon.

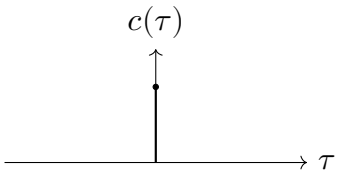
Kalman filtering learned about his filter in Moscow.

We were used at working with analog signals; the leap happened when we began manipulating signals with computers: in order to do this, we need to discretize signals. Analog circuits are different from one another; computers always do the same operations! It's much better for us to manipulate signals in a discrete domain!

What if we have a certain channel with a response which is an impulse?

4.4 Discretized Channel Model

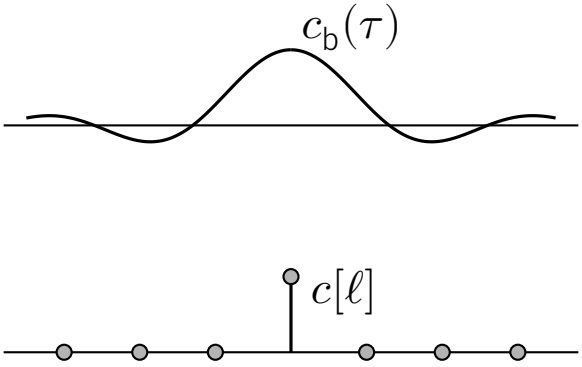
One-tap



Whatever is transmitted, it arrives straight to the receiver. This is true up to a certain point: it might take a while for the signal to travel up to the destination: we might want to move our delta! This is a problem which is called **time synchronization**.

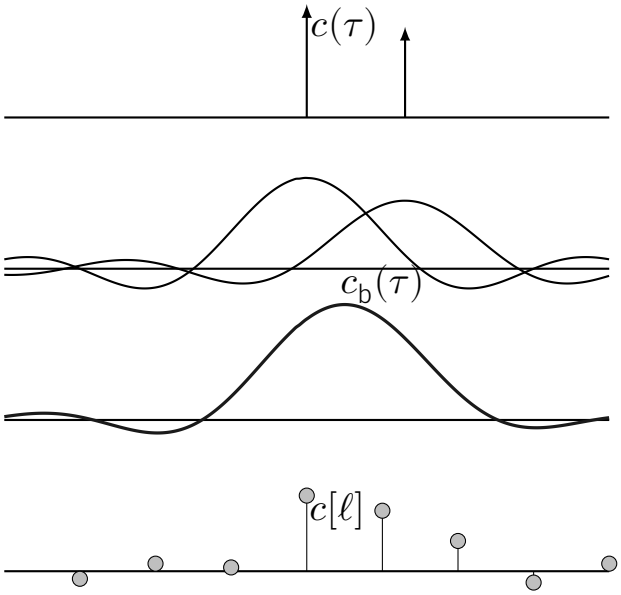
How can the transmitter understand when $t = 0$ is? We might have a form of mismatching if the receiver doesn't understand correctly when the signal started. We might do syncing using autocorrelation: CAZAC sequences are generally used (Constant Amplitude Zero AutoCorrelation waveform).

Here we suppose that our signal is synchronized; let's convolve it with a sinc; we have that sampling it will give a Kronecker delta function.



This only happens if we have synchronization.

Let's look at another channel: we have two paths which will be followed by our signals;



This is a typical example of **reflection**; for each delta, we'll have a sinc; if we sample this, then we'll have four taps (as can be seen in the last figure). In this case what happens is that from two taps, in the discrete domain we got 4.

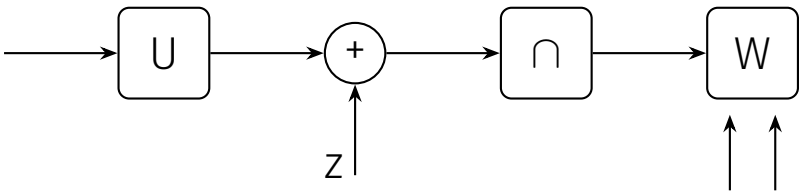
The amount of taps are increasing; we can only sync with one or the other, thus we're always in trouble.

This is a simple problem: if we only have a tap, we'll have a single tap in the frequency domain. If we have multiple taps, stuff might become a nightmare!

Sampling is nice, but we can't do this for free.

Let's make some comments on developing a discrete time model:

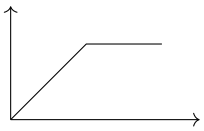
- Noise is still IID and the noise samples are still Gaussian with $\sim \mathcal{N}(0, N_0)$
- We can also use root-raised-cosine filter instead of a sinc filter.
- If we want to detect a signal, it's much easier to do so if we have a white specter: if the noise goes through certain blocks, though, it will no longer be flat!



we can add a **block which whitens the noise**; this is good since then all the subsequent equalization and such work much better. Whitening a noise can be much more difficult.

Pulse shaping: if in the frequency domain we have a sinc, this is optimal. This is a sinc tho, which decays slowly in the time domain. This is why we're using a root-raised-cosine filter: it will have a wider bandwidth but it will be better in the time domain.

Instead of sampling at the Nyquist rate (which allows for perfect recovering), we can say: "what if I sample the signal more frequently? Maybe I can make algorithms which are working better". From the Information theory FOV, Nyquist suffices; but if we have to do with practical issues like non linearities, stuff might be different. Amplifiers are generally non-constant: there is a certain range in which the amplifier is linear, but there is a certain point at which whichever signal we put at the input of the amplifier, we'll get the exact same output; this is called clipping:



For synchronization, it helps to have oversampling (not in the book!); if the Nyquist rate is just in between the right points, it might not sync correctly.

4.5 Transitioning to MIMO (frequency selective and FIR channel expressions; channel normalization)

We're now going towards MIMO; we have a certain number N_t of transmitting antennas and N_r of receive antennas; those numbers can def be different.

We have at the input the value $\mathbf{x}$ and at the output $\mathbf{y}$ with noise $\mathbf{v}$.

Let's start from the causal and FIR **frequency flat** channel; we'll have an output of:

$$\mathbf{y}[n] = \sqrt{\rho}\mathbf{H}\mathbf{x}[n] + \mathbf{v}[n]. \tag{2.81}$$

$\mathbf{H}$ is a matrix such that the impulse response between transmit antenna j and receive antenna i will be represented by $\mathbf{H}_{i,j}$.

◇ Frequency flat (also found as frequency-flat)

By frequency flat, the book refers to a channel which has just one tap, aka $L = 0$.

$\mathbf{x}$ is multiplied by $\mathbf{H}$; this means that the **length of $\mathbf{x}$ is N_t**; the length of $\mathbf{y}$ is N_r.

Skipped

Professor skipped the beginning of **section 2.3.1**, which tackles a continuous IO MIMO relation.

Just like we saw, we're not certain of having a delta; we might have some other paths! We'll model this (a **FIR channel**) just as we did before:

$$\mathbf{y}[n] = \sqrt{\rho}\sum_{l=0}^{L-1}\mathbf{H}[l]\mathbf{x}[n-l] + \mathbf{v}[n]. \tag{2.80}$$

where $[\mathbf{H}[l]]_{i,j}$ represents the response between transmit antenna j and receive antenna i. This means that the **output vector at time n is also depends on previous symbols**. Each vector is then also weighted by a different matrix, which changes over time.

There is no difference, conceptually, with the scalar case; we only have that we have a matrix instead of a vector.

Before, we had normalized the channel; in this case we sum over all the three indexes; we don't want it to be 1, but instead we'd like to be $1 \cdot \#$ of paths $= \boxed{N_t N_r}$.

We can use the definition of the **Frobenius norm** in order to impose the normalization; while we could impose the summation to be equal to 1, this would not allow to include channel models with power asymmetry across antennas; for this reason, we have that

$$\sum_{l=0}^{L-1} \sum_{i=0}^{N_r-1} \sum_{j=0}^{N_t-1} \mathbb{E}[|H[l]_{i,j}|^2] = N_t N_r. \tag{2.82}$$

which can be rewritten more compactly using the definition of Frobenius norm (given in appendix B.5),

$$\sum_{l=0}^{L-1} \mathbb{E}[||\mathbf{H}[l]||_F^2] = \sum_{l=0}^{L-1} \mathbb{E}[\text{tr}(\mathbf{H}[l]\mathbf{H}^H[l])] \tag{2.83}$$

$$= \sum_{l=0}^{L-1} \mathbb{E}[\text{tr}(\mathbf{H}^H[l]\mathbf{H}[l])] \tag{2.84}$$

$$= N_t N_r. \tag{2.85}$$

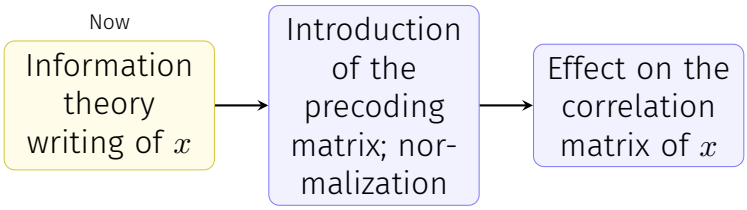
which for the frequency-flat channel simplifies to

$$\begin{aligned} \mathbb{E}[||\mathbf{H}||_F^2] &= \mathbb{E}[\text{tr}(\mathbf{H}\mathbf{H}^H)] \\ &= \mathbb{E}[\text{tr}(\mathbf{H}^H\mathbf{H})] \\ &= N_t N_r. \end{aligned} \tag{2.86}$$

The Frobenius norm is taking matrix and substituting it with the trace of the matrix multiplied with the complex conjugate of said matrix.

4.6 Precoding

4.6.1 Introduction (information theory writing of $x[n]$, matrix F; variation of precoded x)



We have the precoding; we'll start with SISO signals. We'll decompose it in an IT (Information Theory) friendly way:

$$x[n] = \sqrt{E_s P[n]} s[n].$$

where:

- E_s is the average symbol energy
- $P[n]$ is the power allocation at a certain given time
- $s[n]$ are unit-variance symbols (PSK, QAM, Gaussian)

Hello, new writing of $x[n]$! Will you stick around?

Yup. This writing will be reused in chapter 4, when linking OFDM and vector coding (and also when finding the SNR for both).

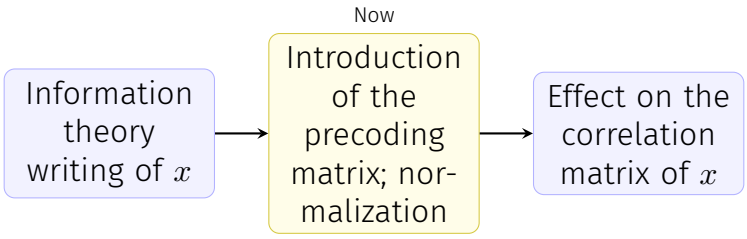
About units...

From what I gather, if we admit that energy is in $W \cdot s$ and power in W, if x is in units of V, then s must have a unit of $V/\sqrt{J}$.

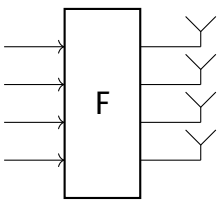
Sending info using a Gaussian: a **Gaussian** is the base we use for the other ways to send signals; we use it as a **bound** for what we can do. If we don't reach the performance of the Gaussian, there is a reason for that.

When we're going from 16 to 64 QAM, we're just making the constellation more dense.

$P[n]$ is the power allocation for each signal; about it:
In our phone we have an algorithm called the *power control algorithm*; when we go away from an antenna, we need to raise the power used to transmit; if we're close, we need to have a forward signal!
What is practically done is that we measure the signal from the base station and we'll know how strong our signal should be.
We'll then change the signal when we go farther from a station.



With MIMO we do *transmit precoding*: we suppose we have $N_s \leq N_t$: the number of transmit symbols (N_s) is at most equal to the number of transmit antennas (N_t). If we're not using an antenna, we can think about what to do with it. If we have 5 symbols but just an antenna, we won't be able to transmit that symbol. **Our vector of symbols can only be transmitted if $N_s \leq N_t$.** In the MIMO we can do more than deciding how much power we should use.



we might decide to use a matrix to mix those symbols; it's a degree of freedom which might prove very useful: why not use it?

For any running symbol, we multiply it with a matrix $\mathsf{F}[n]$ (which can vary with n!!):

$$\mathsf{x}[n] = \sqrt{\frac{E_s}{N_t}}\mathsf{F}[n]\mathsf{s}[n]. \tag{2.96}$$

where:

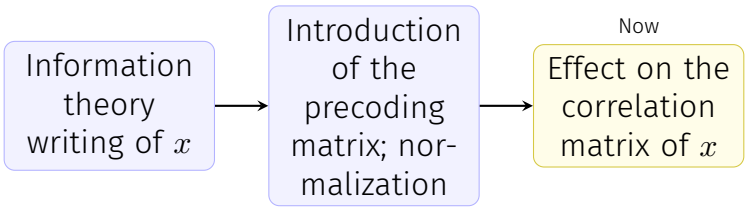
- The $s[n]$ symbols are IID.
- E_s is the average energy per symbol (more about this later).
- $F[n]$ is the precoder matrix; as we'll see, it's normalized in order to respect specific relations. It will have a dimension of $N_t \times N_s$.

We have a restriction on $F[n]$ which uses the **Frobenius norm**; it grants that the antenna is not amplifying the signal:

$$\frac{1}{N}\sum_{n=0}^{N-1} ||\mathsf{F}[n]||_F^2 = \frac{1}{N}\sum_{n=0}^{N-1} \text{tr}(\mathsf{F}[n]\mathsf{F}^H[n]) = N_t. \tag{2.97}$$

Later, we'll need the transmit covariance. $s[n]$ are IID codewords, which means that $\mathbb{E}[\mathsf{s}[n]\mathsf{s}^*[n]] = \mathsf{I}$ (this follows from the fact that independence implies uncorrelation and that these vectors are 0-mean; the main diagonal will have the variance of each signal, which is 1), with dimensions $N_s \times N_s$.

We're always transmitting 0-mean and 1-variance signals.



If we take into account $x[n]$, which has multiplication with the matrix, the covariance will be given by $\mathsf{R_x}$. In order to find its value, we'll apply the fact that $(\mathsf{Fs})^* = \mathsf{s}^*\mathsf{F}^*$. We'll thus have

$$\begin{aligned}\mathsf{R_x} &= \frac{E_s}{N_t}\mathbb{E}[\mathsf{Fss}^*\mathsf{F}^*] \\ &= \frac{E_s}{N_t}\mathsf{F}\mathbb{E}[\mathsf{ss}^*]\mathsf{F}^* \\ &\overset{(*)}{=} \frac{E_s}{N_t}\mathsf{FF}^*.\end{aligned}$$

re-introducing the time dependent notation, we have (for ID s)

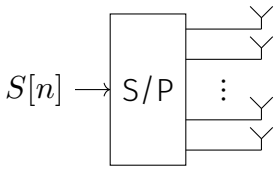
$$\mathsf{R_x}[n] = \frac{E_s}{N_t}\mathsf{F}[n]\mathsf{F}^H[n]. \tag{2.101}$$

Comparison with normal case

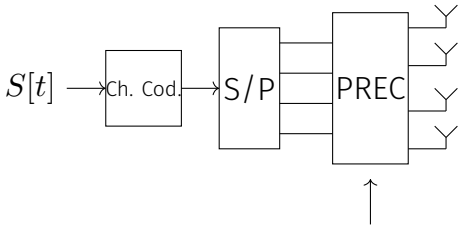
If we were to set $\mathsf{F}[n] = \mathsf{I}$, we'd find that the non-precoded matrix of x is

$$\frac{E_s}{N_t}\mathsf{I}.$$

We have a source of symbols, we pass them through a serial to parallel and then we go through the antennas; usually, we use a code



What we generally do is actually code after the series/parallel transition!



4.7 Recap of singular value decomposition

Let's revisit the concept of singular value decomposition!

This is heavily used in machine learning and such

We have a certain matrix A of dimensions $N \times M$; we can always decompose it as

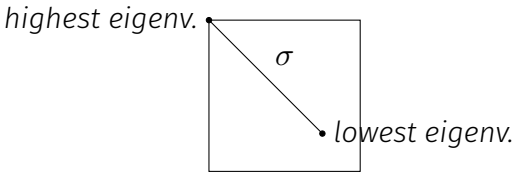
$$\mathsf{A} = \mathsf{U}\boxed{?}\mathsf{V}^*,$$

where:

- U is unitary $N \times N$.
- V^* is unitary $M \times M$.
- $\boxed{?}$ is $N \times M$ with non negative entries on its diagonal ordered from largest to smallest.

The entries of said matrix **are the singular values of the original matrix, aka the square root of the eigenvalues of A**.

The $\boxed{?}$ matrix is generally like



If we choose $\mathbf{u}_j$, a column of $\mathbf{V}$ and $\mathbf{u}_j$, a column of $\mathbf{U}$, we'll have

$$\mathbf{A}\mathbf{v}_j = \sigma_j \mathbf{u}_j$$
$$\mathbf{A}^*\mathbf{u}_j = \sigma_j \mathbf{v}_j$$

where σ_j is the j-th element on the diagonal of $\boxed{?}$.

We just need to remember the dimensions in order to have this work correctly.

Property of unitary matrices

For a unitary matrix, we have that

$$\mathbf{U}^{-1} = \mathbf{U}^*.$$

This is why unitary matrices are very useful.

$\mathbf{U}$ is very easy to invert and same goes for $\mathbf{V}$. Then we might say: it will be easy to invert $\mathbf{A}$.

$\mathbf{A}$ will also have the following properties:

Table B.1 Bases for the column, row, and null spaces of an $N \times M$ matrix $\mathbf{A} = \mathbf{U}\boxed{?}\mathbf{V}^*$

Subspace	Dimension	Basis
Column space of $\mathbf{A}$	$r = \text{rank}(\mathbf{A})$	First r columns of $\mathbf{U}$
Row space of $\mathbf{A}$	r	First r columns of $\mathbf{V}$
Null space of $\mathbf{A}^T$	$(N-r)$	Last $(N-r)$ columns of $\mathbf{U}$
Null space of $\mathbf{A}$	$(M-r)$	Last $(M-r)$ columns of $\mathbf{V}$

About the number of streams

If we have 1 transmitter and 1 receiver, we know many things; but we didn't consider a channel which is dispersive and generates ISI; how do we deal with ISI?? This is handled in section 2.5 of the book.

Let's suppose, at least for now, that we have no ISI; the Shannon bound tells us that we have a maximum amount of bits per symbol per hertz which we can put through the channel.

People said: we want to transmit even more!! In order to do so, we increase the number of transmitter and receiver antennas. Let's say we have 3 transmitters and 5 receivers; the interactions between all of these won't be described by a vector anymore, but rather by a matrix. We'll see that we have **a number of independent streams which is**

$$\#\text{streams} \leq \min\{N_t, N_r\};$$

we use MIMO since we want to be able to transmit even more than what would have been possible while respecting the Shannon bound. It's useless to have 1 transmitter and many receivers: we'd be able to only have 1 stream.

It's difficult to have many antennas in the phone, much more than it is to have many in our station.

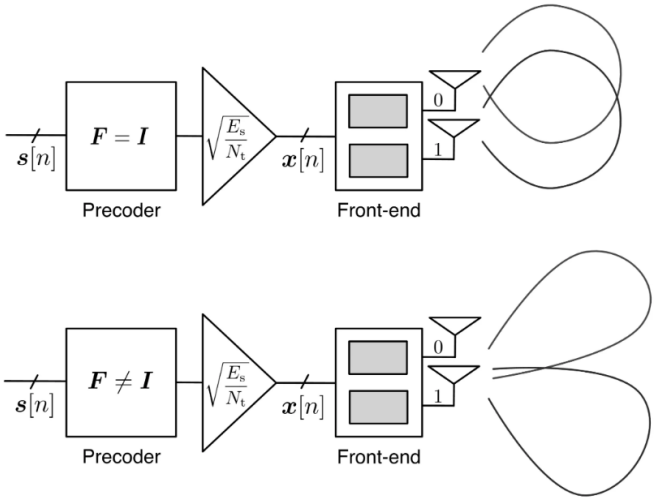
Before looking at this, we want to understand if this works at all, though! We first need to know the theory and that this is possible and only then will we be able to use this.

4.8 Applying precoding: decomposing F and explaining the various parts of its new form

Let's apply what we've learned to the model

$$\mathbf{x}[n] = \sqrt{\frac{E_s}{N_t}}\mathbf{F}[n]\mathbf{s}[n], \tag{2.98}$$

we know we have $\mathbf{s}[n]$ at the input and let's suppose that $\mathbf{F} = \mathbf{I}$: we go directly in the antennas. If we do this, all the antennas transmit all like a circle (this depends on how they're built), as it comes. If we then have $\mathbf{F} \neq \mathbf{I}$, we can get each antenna to point to a specific user; this allows us to maximize the gain we have from the transmit antenna.



What we want to do is to multiply our symbols with a matrix and do something. What can we achieve? In order to better understand this, we can use singular value decomposition to decompose $\mathbf{F}$ as:

$$\mathbf{F}[n] = \mathbf{U}_F[n]\boxed{?}_F[n]\mathbf{V}_F^H[n], \tag{2.100}$$

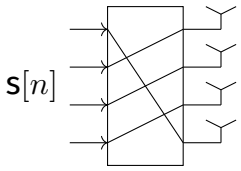
where:

- $\mathbf{V}$ is the **mixing matrix**, which is a $N_s \times N_s$ unitary matrix
- $\boxed{?}$ is the **power allocation matrix** and is $N_t \times N_s$ (more about this later)
- $\mathbf{U}$ is the **beam steering matrix**, which is a $N_t \times N_t$ unitary matrix

If we had only

$$\mathbf{V}_F^*\mathbf{s},$$

we'd get N_t symbols; this mixes the various signals towards the various antennas:



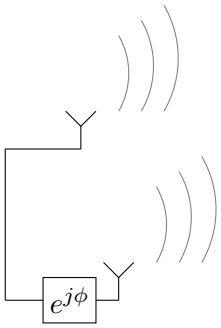
By only doing this we'd create confusion; this is why we also have a power allocation matrix: it has a diagonal which expresses how much power we'll give to each antenna. It will have a shape like

$$\boxed{?}_F[n] = \begin{bmatrix} \mathsf{P}^{\frac{1}{2}} \\ \mathsf{0} \end{bmatrix} \tag{2.101}$$

where P is a $N_s \times N_s$ matrix which will be defined below.

What's really important is U_F, which allows us to **steer the signal**.

This had been discovered a long time ago! If we have a radar and we transmit with a phase shift on one antenna:



In a certain direction we'll have a specific ϕ; if $\phi = \pi$, we're not transmitting anything. If we look at another direction, we can tilt an antenna electronically to transmit over a certain direction.

This is used by radars, even though radars transmit an impulse and want to get stuff back. In order to understand where the signal is coming from they use multiple antennas and they use something like a steering matrix. If we take the nose of an airplane, the whole nose of an airplane has a radar in it.

It's nothing new in this sense!

So now we can have different directions from an array of antennas. By using appropriate matrices, we can make spatial streams.

Our tools for designing are U_F and the amount of power we put in each antenna (the sum of the power cannot be more than N_t), expressed by the matrix

$$\mathsf{P} = \text{diag}(P_1, \dots, P_{N_s - 1})$$

such that $\sum_{i=0}^{N_s-1} P_i = N_t$.

The notation P_j is especially interesting since the jth mixed signal (aka $[\mathsf{V}_F^H \mathsf{s}[n]]_j$) will have a power of $\frac{E_s}{N_t} P_j$ allocated to it, where the E_s/N_t term comes from our definition in (2.98).

The jth amplified signal will be launched from all transmit antennas and will be weighted with the coefficients in the jth column of $\mathsf{U}_F[n]$.

If we were to choose a silly (aka useless) setup, we might have both $\mathsf{U}_F, \mathsf{V}_F$ be identity matrices and $\mathsf{P} = \frac{N_t}{N_s}\mathsf{I}$, which entails

$$\mathsf{F}[n] = \sqrt{\frac{N_t}{N_s}} \begin{bmatrix} \mathsf{I}_{N_s} \\ \mathsf{0} \end{bmatrix}. \tag{2.102}$$

With such trivial precoder, each of N_s antennas directly radiates one of the data streams within $\mathsf{s}[n]$ while the remaining $N_t - N_s$ antennas are silent; the transmit signals are independent. If this is the case, the transmission is said to be *isotropic* from a precoding pov or even *unprecoded*.

Example 2.10

Let $N_t = N_s = 2$ and suppose that the two signals within $s[n]$ are QPSK-distributed. Examine the structure of the transmit signal $x[n]$ produced by a time-invariant precoder.

Solution

Dropping the dependence on n, let $\mathsf{P} = \mathrm{diag}(P_1, P_2)$ and

$$\mathsf{U}_F = \begin{bmatrix} U_{00} & U_{01} \\ U_{10} & U_{11} \end{bmatrix}, \qquad \mathsf{V}_F = \begin{bmatrix} V_{00} & V_{01} \\ V_{10} & V_{11} \end{bmatrix}. \tag{2.103}$$

The application of the mixing matrix to s yields the mixed signal vector

$$\mathsf{V}_F^*\mathsf{s} = \begin{bmatrix} V_{00}^*(\mathsf{s})_0 + V_{10}^*(\mathsf{s})_1 \\ V_{01}^*(\mathsf{s})_0 + V_{11}^*(\mathsf{s})_1 \end{bmatrix} \tag{2.104}$$

whose unit-variance entries conform to 16-ary distributions, in general distinct. Then, after power allocation,

$$\sqrt{\frac{E_s}{2}}\mathsf{P}^{1/2}\mathsf{V}_F^*\mathsf{s} = \begin{bmatrix} \sqrt{\frac{E_s P_1}{2}}(V_{00}^*(\mathsf{s})_0 + V_{10}^*(\mathsf{s})_1) \\ \sqrt{\frac{E_s P_2}{2}}(V_{01}^*(\mathsf{s})_0 + V_{11}^*(\mathsf{s})_1) \end{bmatrix}. \tag{2.105}$$

The top signal is launched from the two antennas, weighted by U_{00} and U_{10}, while the bottom signal is launched weighted by U_{01} and U_{11}. If $\mathsf{F} = \mathsf{I}$, then each signal is radiated from only one antenna. Conversely, if $\mathsf{F} \neq \mathsf{I}$, each signal is transmitted from both antennas and steered in a certain direction. Both possibilities are illustrated in Fig. 2.11.

Comments:

- The $*$ operator on matrix also implies transposition
- A unitary matrix is just rotating the vector: it won't change its magnitude
- If $\mathsf{F} = \mathsf{I}$, each signal is radiated by one antenna
- If $\mathsf{F} \neq \mathsf{I}$, both signals are transmitted by both antennas and we can steer the antennas.

4.9 Signal power constraints

Which power constraint will we enforce? We might have average energy per block or per symbol or even per antenna:

- **Per-block** $\frac{1}{N}\sum_n ||\mathsf{x}[n]||^2 \leq P$, or equivalently $\frac{1}{N}\sum_n \text{tr}(\mathsf{R}_\mathsf{x}[n]) = P_t$.

  $\rightarrow$ Then the precoder $\frac{1}{N}\sum_n ||\mathsf{F}[n]||_F^2 = N_t$

- **Per-symbol** $\mathbb{E}[||\mathsf{x}[n]||^2] = E_s$ or $\text{tr}(\mathsf{R}_\mathsf{x}[n]) = E_s$

  $\rightarrow$ Then the precoder satisfies $\text{tr}(\mathsf{F}[n]\mathsf{F}^H[n]) = \text{tr}(\mathsf{P}[n]) = N_t$

- **Per-antenna** $(\mathsf{R}_\mathsf{x})_{ii} = \frac{E_s}{N_t}$, which means that

  $||\mathsf{F}[n]||_{i,:}^2 = 1$ for $i = 0, \dots, N_t - 1$

We might even be super strict and say *every symbol should have the same energy*, or more relaxed and say: on average a block should have a certain amount of energy. What we have depends on our situation.

Examples

Regulation says that if we put a sphere around the top of the antenna; the power of the outgoing em field should be below a certain value. If we have that power, we'll need to share it between antennas.
If we have DAS (Distributed Antenna System), which is now used in tunnels, we can put the same power on the various antennas.

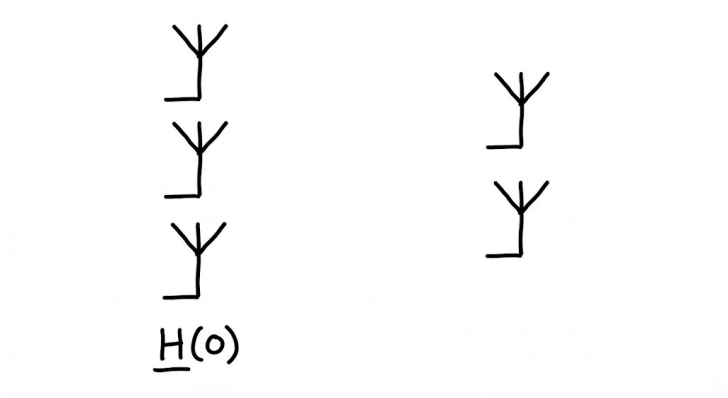
The constraint depends on the practical situation which we have.

4.10 Vectorizing the channel: MIMO multipath channel (specifically, stacked notation absorbing convolution, time dependency and MIMO) for receiver i and for the whole system

We were looking at how to vectorize the channel; this paragraph, specifically, investigates how to reach for MIMO a result similar to the (2.74) we reached for SISO. We also include previous values of the signal since the channel has memory. This is like convolution, but in the time domain.

Now, instead of having SISO systems, we can jump to the MIMO case. In summary, instead of having scalars in the matrix H, in the MIMO system we'll have matrices.

$H(0)$ tells how the signal gets from one part to the other. In the SISO case we had $h[k], k = 0, \dots, L$: we had L paths since we weren't in a flat channel. We won't stop at considering only the matrix corresponding to $h[0]$; we'll also look at the matrix for the system's memory.



If we then consider the receiver antenna i, we'll have that the received signal will be the sum of all the values:

$$y_i^{(r)} = \sqrt{\rho} \sum_{j=0}^{N_t-1} [\mathsf{H}]_{i,j} \mathsf{x}_j^{(N+L)} + \mathsf{v}_i^{(r)}. \tag{2.89}$$

where H is a matrix of matrices and $N+L$ is there since we have L delays. The output does not only depend on one input vector, but it's still a vector of dimension $N+L$ depending on the input of each transmit antenna ($x_j^{(N+L)}$).

What are those $N, N+L$ subscripts?

Those subscripts represent the dimensions of vectors and matrices. Just as an example, x is a $(N+L)\times 1$ vector and H is a $N\times(N+L)$ matrix.
Specifically, then, each vector contains the time information for signals for $n = 0, \dots, N-1$, with the relation Hx representing the convolution operation.

We're considering the antenna i:

SISO!

We have many antennas transmitting to i; we need to sum over all the transmitting antennas and over all the vectors transmitted at the j-th antenna. The jump we're doing with this new equation is given by the fact that we have many transmission antennas! y will have a dimension of $N_r N$! We're putting the various vectors one on top of the other:

$$\bar{y}_{N_r N} = \begin{bmatrix} \bar{y}_N^{(0)} \\ \vdots \\ \bar{y}_N^{(N_r-1)} \end{bmatrix}.$$

$$\bar{x}_{N_t(N+L)} = \begin{bmatrix} \bar{x}_{N+L}^{(0)} \\ \vdots \\ \bar{x}_{N+L}^{(N_t-1)} \end{bmatrix}.$$

Before we had $N, N+L$ on the matrix; now we have $N_r N, N_t(N+L)$. The input vector is $N_t(N+L)$: we're stacking the vectors of the input antennas.

All the matrices in H are $N, N+L$ matrices:

$$\begin{bmatrix} \bar{H}_{N,N+L}^{(0,0)} & \bar{H}_{N,N+L}^{(0,1)} & \cdots & \bar{H}_{N,N+L}^{(0,N_t-1)} \\ \bar{H}_{N,N+L}^{(1,0)} & \bar{H}_{N,N+L}^{(1,1)} & \cdots & \bar{H}_{N,N+L}^{(1,N_t-1)} \\ \vdots & \vdots & \ddots & \vdots \\ \bar{H}_{N,N+L}^{(N_r-1,0)} & \bar{H}_{N,N+L}^{(N_r-1,1)} & \cdots & \bar{H}_{N,N+L}^{(N_r-1,N_t-1)} \end{bmatrix}$$

Each of the matrices is like a SISO matrix; we're considering how the information transmitted from antenna j is arriving at antenna i.

In summary, we can express the output of the MIMO system as

$$\bar{y}_{N_r N} = \sqrt{G}\bar{H}_{N_r N, N_t(N+L)}\bar{x}_{N_t(N+L)} + \bar{v}_{N_r N}. \tag{2.95}$$

By the way, L is the **maximum length of the interference channel**; if we have two antennas with a channel with $L_1 = 3$ and another with $L_2 = 7$, we'll use $L = 7$. We'll just put zeros for all the paths which an antenna does not have:

$$h(0), h(1), h(2), 0, 0, 0, 0.$$

If we want to have a well-formed matrix, we select L as the maximum of all the Ls.

The one we got is a compact form, which is the same as the one we had before (with $\bar{y}$), just with bigger dimensions.

By the way, book uses terms *column* and *fat* matrices.